</> codigo.c

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1s >

É malware: um rootkit de kernel (LKM rootkit), especificamente uma versão do Diamorphine.

Análise passo a passo

1. **Tipo de código e estrutura geral**
   É um Loadable Kernel Module (LKM) para Linux. Usa headers de kernel (`linux/module.h` , `linux/sched.h` , `linux/syscalls.h` , etc.), tem `module_init` / `module_exit` , e declara `MODULE_LICENSE` , `MODULE_AUTHOR` e `MODULE_DESCRIPTION("LKM rootkit")` . O autor é "m0nad". Isso já indica que não é um driver legítimo.
2. **Localização da tabela de syscalls e resolução de símbolos**
   O código obtém a `sys_call_table` (via `kallsyms_lookup_name` ou busca por força bruta em kernels mais antigos). Também trata proteções de memória (CR0 no x86, `update_mapping_prot` no ARM64) para poder modificar a tabela de syscalls. Em kernels ≥ 6.9 no x86 usa uma técnica de "flipswitch" para redirecionar chamadas. Isso é típico de rootkits que precisam hookar syscalls.
3. **Hooks em syscalls**
   Substitui três syscalls:
   - `getdents` / `getdents64` → filtragem de entradas de diretório.
   - `kill` → tratamento especial de sinais.
4. **Funcionalidade de ocultação de arquivos/processos (`hacked_getdents` / `hacked_getdents64`)**
   - Copia o buffer de dirent para o kernel.
   - Se o diretório for `/proc` (identificado pelo inode), verifica se o PID correspondente tem a flag `PF_INVISIBLE` e remove a entrada.
   - Em outros diretórios, remove qualquer entrada cujo nome comece com `MAGIC_PREFIX` .
   - Reescreve o buffer de volta para o usuário, encurtando o tamanho retornado.
     Resultado: arquivos, diretórios e processos ficam invisíveis para `ls` , `ps` , `find` , etc.
5. **Funcionalidade de privilégio e controle via sinais (`hacked_kill`)**
   Intercepta a syscall `kill` e trata sinais especiais (definidos em `diamorphine.h` , não mostrado, mas usados como `SIGINVIS` , `SIGSUPER` , `SIGMODINVIS`):
   - Alterna a flag `PF_INVISIBLE` de um processo (torna-o oculto/visível).
   - Chama `give_root()` , que eleva as credenciais do processo atual para uid/gid 0 (root).
   - Alterna a visibilidade do próprio módu
     `module_show`).

Copied link to clipboard. Shared links can be viewed by anyone with the link.

6. **Ocultação do próprio módulo**
   No `init` o módulo se remove da lista de módulos (`list_del`) e limpa atributos de seção (`tidy`). Isso impede que apareça em `lsmod` / `/proc/modules` enquanto estiver ativo.
7. **Elevação de privilégio (`give_root`)**
   Usa `prepare_creds` + `commit_creds` (ou manipulação direta de `uid` / `gid` em kernels muito antigos) para conceder root completo ao processo que enviar o sinal especial.
8. **Compatibilidade com várias versões de kernel**
   Contém inúmeros `#if LINUX_VERSION_CODE` para funcionar desde kernels antigos até 6.9+, cobrindo mudanças em estruturas (`pt_regs` , `cred` , `files_struct` , etc.). É código de rootkit mantido e adaptado.

Conclusão

O arquivo implementa exatamente as capacidades clássicas de um **rootkit de kernel**:

- Ocultação de processos e arquivos.
- Ocultação do próprio módulo.
- Elevação de privilégio sob demanda.
- Hook de syscalls com proteção de memória removida temporariamente.

Não há funcionalidade legítima (driver de dispositivo, filesystem, etc.). A própria descrição do módulo declara “LKM rootkit”.

Classificação final: malware (rootkit).

Qual é a categoria principal deste malware?

Rootkit (mais especificamente, um **LKM rootkit** / rootkit de kernel).

Essa é a categoria principal. O código atua no espaço do kernel (módulo carregável), faz hook de syscalls (`getdents` / `getdents64` e `kill`), oculta processos, arquivos e o próprio módulo, e oferece elevação de privilégio sob demanda. Essas são as características definidoras de um rootkit de kernel no Linux.

↘ É ransomware?

↘ Como ele esconde os processos?

↘ É malware de ransomware?

Copied link to clipboard. Shared links can be viewed by anyone with the link.